LxDigitizer Sample Timing Design

The LxDigitizer is a system of components designed to digitize lightning signals sampled at 1,024,000 samples per second, and write these signals to SD storage, along with the precise time of acquisition of each signal. This document describes a method of providing high resolution time data that corresponds to acquired lightning signal data.

System components

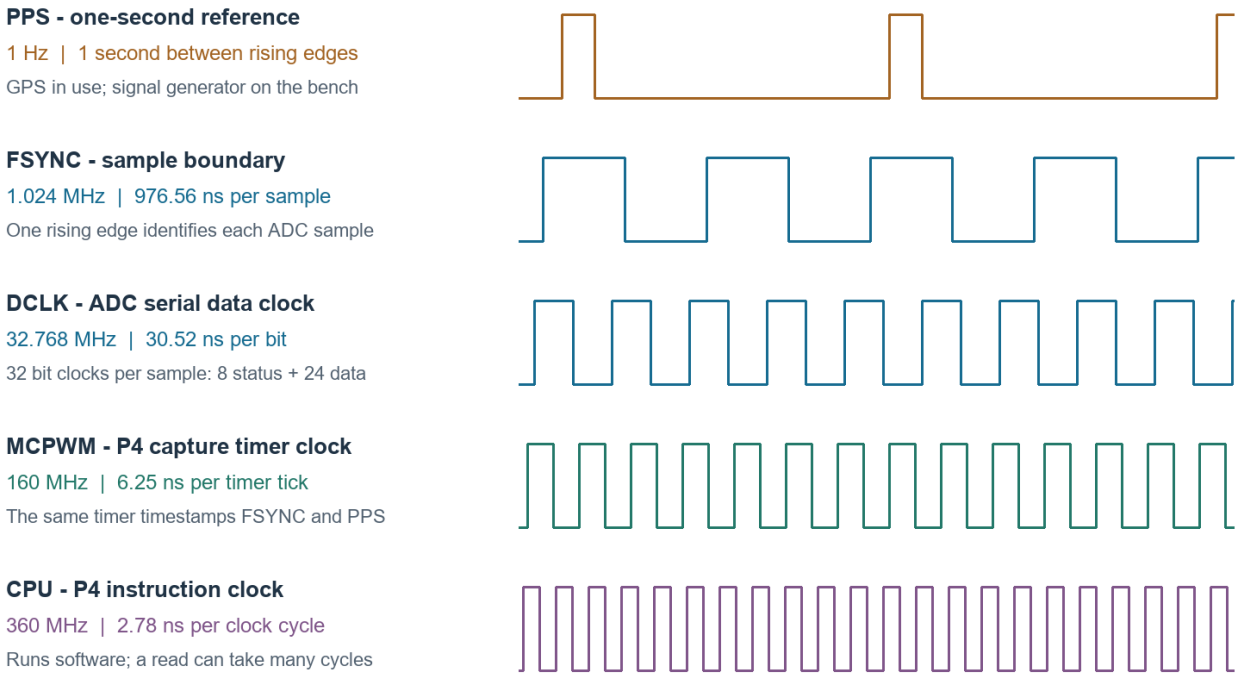
The LxDigitizer system includes the following components:

- A lightning signal source that produces an analog signal
- A GPS unit that supplies time and location data
- A 24-bit ADC, currently an ADS127L18, that digitizes the lightning signal
- An ESP32-P4 microcontroller, which acquires the digitized Lx Signal and GPS time pulses, and writes data to SD storage
- A voltage translator, which translates digitizer 1.8 VDC to 3.3 VDC required by the P4
- An SD storage device

Challenges to acquiring signal timing

System clocks and timing signals

Each row uses its own time scale. Pulse widths and edge positions are illustrative.



Nominal rates for our current configuration. Diagram does not specify clock-source or phase relationships.

Each saved sample needs a time that represents **when the analog signal was measured**, rather than when software received or stored the result. Several challenges must be addressed:

- **Independent clocks.** The GPS supplies a pulse per second (PPS). The digitizer supplies an FSYNC pulse identifying each sample's output frame. These pulses are not synchronized with each other or with the P4 capture timer.
- **Sample spacing.** At 1,024,000 samples per second, successive FSYNC pulses are approximately **977 ns apart**. A PPS edge can occur anywhere between them. Identifying the sample nearest PPS is therefore insufficient for timing resolution substantially finer than one sample period. We must also measure the time offset between PPS and FSYNC.
- **Digitizer delay.** The analog signal passes through the ADC and its digital filter before the corresponding FSYNC reaches the P4 through the voltage translator. Our calibration established **2,553 ns as the correction to subtract from the FSYNC time** for the tested configuration.
- **Clock drift.** The actual digitizer and capture-timer frequencies can differ slightly from their nominal values and change over time. Sample times must account for these differences rather than assume that every second contains exactly 160 million capture-timer ticks or 1,024,000 samples.
- **Continuous throughput.** The system must receive, decode and store approximately **24.6 million sample-data bits per second**, while maintaining the association between each sample and its timing information. Buffering and SD write delays must not change that association or cause sample loss.

Design overview

The design ties each sample to GPS time by measuring FSYNC and PPS with **the same P4 hardware timer**. This timer is part of **MCPWM (Motor Control Pulse Width Modulator)**, a hardware peripheral available on the ESP32-P4. Although intended for motor control, MCPWM also provides an input-capture function that saves timestamps when external signals change. In this design, its capture timer runs at 160 MHz, giving a tick interval of **6.25 ns**.

When an FSYNC or PPS rising edge arrives, hardware saves the timer's tick count. Software reads that saved timestamp later. Consequently, software response time does not determine the timestamp.

The solution combines three pieces of information:

- **GPS date and time**, received through UART, identifies the UTC second associated with a PPS pulse.
- **PPS timestamps** establish the relationship between timer ticks and elapsed seconds.
- **Numbered FSYNC timestamps** connect the timer readings to specific digitized samples.

Samples are grouped into **sBlocks**, one for each PPS interval. Each sBlock is preceded by a **tBlock** containing the first and last sample's FSYNC timestamps, the opening and closing PPS timestamps, and the sample count. The file header supplies the UTC starting time.

On readback, these values provide the time of each sample's FSYNC. Subtracting the calibrated **2,553 ns** delay gives the estimated time of the analog input represented by that sample. Using measured tick differences accounts for the average clock rates during each interval.

The timer's **6.25 ns resolution is not a guarantee of 6.25 ns absolute accuracy**. Absolute accuracy also depends on the GPS reference, delay calibration and hardware capture timing.

Binary file structure

Each file begins with a **256-byte header**, followed by repeated **tBlock/sBlock pairs**:

```
[File header]
[tBlock 0][sBlock 0]
[tBlock 1][sBlock 1]
...
[End marker]
```

The header contains the UTC starting time and information needed to interpret the file.

Each **48-byte tBlock** contains timing information for the sBlock immediately following it. The **sBlock** contains consecutive samples, stored as three bytes per sample. Its sample count is specified in the tBlock and can vary between blocks.

Each pair corresponds to one PPS interval. The exact sample boundaries are established using FSYNC timestamps, as described in the implementation section.

All multibyte integers are stored **least-significant byte first**, called *little-endian*. Samples contain no embedded timing bits.

The file structure is independent of the **128-KiB chunks** used for SD writes. A chunk can contain portions of several records; it does not define a sample or timing boundary.

Header

The header establishes the time origin and format for the entire file. Offsets below are byte positions measured from the beginning of the file.

Offset	Bytes	Field	Meaning
0	2	`fileId`	The two characters `DG`, identifying this file format
2	2	`version`	Format version; currently 1
4	8	`utcFirst`	UTC time of the opening PPS for the first sBlock, expressed as whole seconds since January 1, 1970
12	4	`delayNs`	Delay correction subtracted from sample FSYNC times; currently 2,553 ns
16	4	`blockPairsPerFile`	Maximum number of tBlock/sBlock pairs in this file; default 60
20	236	Reserved	Set to zero; available for future format additions

`utcFirst` provides the absolute time reference. Each tBlock's `nSec` specifies how many whole seconds its opening PPS follows that reference.

`blockPairsPerFile` specifies the configured limit, not the number actually written. A recording stopped normally can contain fewer pairs.

tBlock

Each 48-byte tBlock supplies the timing information for the sBlock immediately following it. Offsets are measured from the beginning of the tBlock.

Offset	Bytes	Field	Meaning
0	8	`tcFsFirst`	FSYNC tick count for the first sample in the sBlock
8	8	`tcFsLast`	FSYNC tick count for the last sample in the sBlock
16	8	`tcPpsFirst`	Tick count of the opening PPS
24	8	`tcPpsLast`	Tick count of the closing PPS
32	4	`sCount`	Number of samples in the following sBlock
36	2	`nSec`	Whole seconds from the file's `utcFirst` to this block's opening PPS
38	10	Reserved	Set to zero

The four tick counts are unsigned 64-bit integers. `sCount` is an unsigned 32-bit integer, and `nSec` is an unsigned 16-bit integer.

The first and last FSYNC timestamps, together with `sCount`, establish the average spacing between samples. The two PPS timestamps establish how many timer ticks represent one second during that interval.

Adjacent tBlocks share a PPS timestamp: one block's `tcPpsLast` equals the next block's `tcPpsFirst`. Their FSYNC timestamps refer to **different, consecutive samples**: one block's last sample immediately precedes the next block's first sample.

Each tBlock contains enough information to calculate its samples' times using the file header, without reading another tBlock.

sBlock, completion and file rotation

An sBlock contains `sCount` consecutive samples. Each sample is a signed 24-bit value stored in three bytes, least-significant byte first. ADC status bits are not stored.

The sBlock occupies exactly `3 × sCount` **bytes**. The next tBlock begins immediately afterward.

File completion. A 48-byte, all-zero record at the next tBlock position marks the end of recorded data. Zero-valued samples are ordinary data; the reader finds the end marker by following block lengths, not by searching for zeros.

File rotation. By default, each file holds up to 60 tBlock/sBlock pairs, approximately one minute of samples. The system uses ten file slots, returning to overwrite the oldest file when all slots have been used. Both limits are configurable.

Each new file has its own UTC starting time, and its first tBlock has `nSec = 0`.

Only complete PPS intervals are retained. Partial intervals at startup and shutdown are omitted, so a normally stopped recording may end with fewer than 60 pairs in its final file.

Calculating each sample's time

Number the samples within an sBlock starting at zero: `iSample = 0` is the first sample, and `iSample = sCount - 1` is the last.

First calculate the average number of timer ticks between samples:

```
dtcSample = (tcFsLast - tcFsFirst) / (sCount - 1)
```

There are `sCount - 1` intervals between the first and last samples. For example, 100 samples have 99 intervals.

Calculate the desired sample's FSYNC tick count:

```
tcSample = tcFsFirst + iSample × dtcSample
```

Next calculate the number of timer ticks between the opening and closing PPS pulses:

```
dtcPps = tcPpsLast - tcPpsFirst
```

These PPS pulses are one second apart. Dividing a tick difference by `dtcPps` therefore converts it to seconds:

```
tFs = utcFirst + nSec + (tcSample - tcPpsFirst) / dtcPps
```

Finally, convert the FSYNC time from seconds to nanoseconds and subtract the digitizer delay:

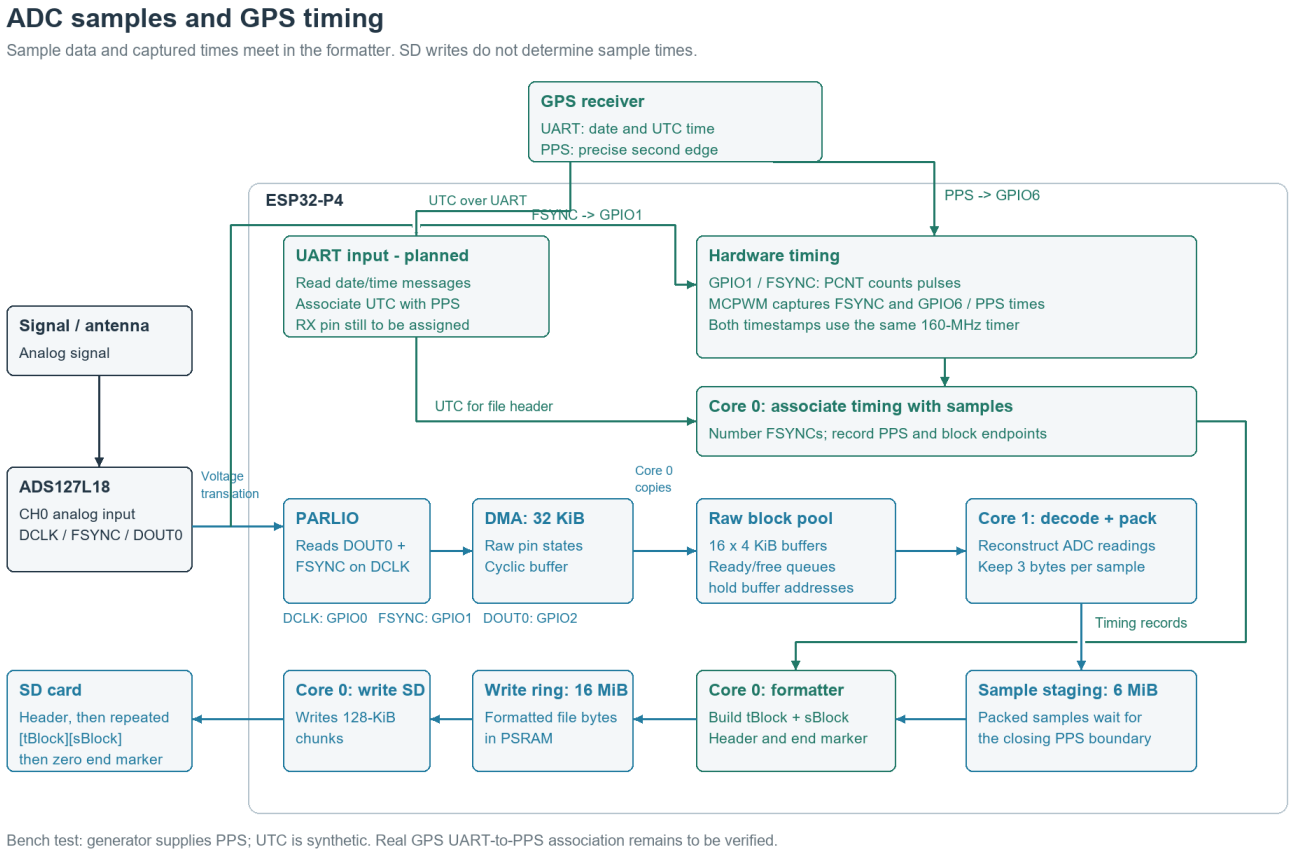
```
tSample = tFs × 1,000,000,000 - delayNs
```

These equations describe the mathematical calculation. In software, keep whole UTC seconds separate from the fractional time until the final conversion to nanoseconds. Combining them into an ordinary floating-point timestamp can lose precision larger than the 6.25-ns timer tick.

`tSample` is the estimated UTC time of the analog input represented by the sample, expressed in nanoseconds since January 1, 1970.

The calculations distribute sample times evenly between the measured FSYNC endpoints and use the measured PPS interval to convert ticks to seconds. They account for average clock rates during the interval, but not individual clock jitter or rate changes within it.

Implementation details: associating a captured FSYNC with a decoded sample



Startup establishes the sample-number convention

At the start of acquisition, the ADC is stopped while the P4 prepares to receive data and resets the pulse counters. Acquisition then begins.

Each normal sample frame contains **32 DCLK cycles: eight status bits followed by 24 sample bits**. In the observed startup sequence, the first captured frame contains only 31 cycles. The receiver discards that first frame and numbers subsequent samples as follows:

```
FSYNC pulse 1 → discarded startup frame
FSYNC pulse 2 → sample 0
FSYNC pulse 3 → sample 1
...
sample index = FSYNC pulse number - 2
```

This establishes the connection between the FSYNC pulse count and the position of each sample in the received data.

Sample numbering continues throughout acquisition, including across file changes. The first sample retained in a file is generally later than acquisition sample zero because recording waits for a complete PPS interval.

Connections

Signal	P4 connection	Purpose
ADC DCLK, through voltage translator	GPIO0	Clock for receiving serial sample bits
ADC FSYNC, through voltage translator	GPIO1	Identifies sample frames; supplies pulse counting and timestamp capture
ADC DOUT0, through voltage translator	GPIO2	Carries CH0 sample data and status bits
GPS PPS	GPIO6	Supplies the one-second timing reference
GPS serial output	UART receive input; pin to be assigned	Supplies UTC date and time

FSYNC and PPS use separate MCPWM capture channels sharing the same timer, so their timestamps can be compared directly.

Two hardware peripherals provide the observations

The FSYNC signal enters GPIO1 and feeds two hardware peripherals:

- **PCNT counts FSYNC rising edges.** Its register holds the pulse count.
- **MCPWM saves the timer tick count at each FSYNC rising edge.** Its capture register holds the timestamp of the most recent edge. MCPWM does **not** save a pulse number.

Software reads these registers separately. An FSYNC edge can occur between the reads, so the pulse count and timestamp might refer to different pulses. They are not automatically a matched pair.

To establish the starting reference, software reads:

```
pulse count → timestamp → pulse count again → timestamp again
```

It accepts the reference only when both pulse-count readings agree, both timestamp readings agree, and the reads satisfy the timing checks. The checks require the read sequence to take less than one sample period and the reference used to extend the hardware pulse counter to be no more than 100 µs old.

This supplies the starting pulse number and timestamp used by the calculation in the next section.

Frequent observations maintain numbering

Software reads the latest captured FSYNC timestamp roughly every millisecond, rather than only once per PPS. It does not need to read every FSYNC timestamp.

Starting from the previous numbered timestamp, software calculates how many pulses have elapsed:

```
deltaTicks = current timestamp - previous timestamp
deltaPulses = round(deltaTicks / 156.25)
predicted pulse number = previous pulse number + deltaPulses
```

The value **156.25** is the nominal number of capture-timer ticks per sample:

```
160,000,000 ticks/second ÷ 1,024,000 samples/second
    = 156.25 ticks/sample
```

For example:

```
Previous pulse number:       1000
Previous timestamp:       200,000 ticks
Current timestamp:        360,000 ticks

Elapsed ticks:            160,000
Elapsed pulses:           160,000 / 156.25 = 1024
Predicted pulse number:    1000 + 1024 = 2024
```

The new timestamp and calculated pulse number become the reference for the next calculation. PCNT supplies an independent check of that number, as described next.

The implementation uses equivalent integer arithmetic to perform this calculation efficiently.

Checking the calculated number

Whenever the PCNT count and captured timestamp pass the read-consistency and timing checks described above, software compares the hardware pulse count with the predicted pulse number.

For example:

```
Predicted pulse number: 2024
PCNT pulse count:       2024 → agreement; continue

Predicted pulse number: 2024
PCNT pulse count:       2023 → disagreement; fault and stop recording
```

Software never changes the predicted number merely to make it agree with PCNT.

If a PCNT observation fails the read checks, it is unavailable for this comparison; that alone is not a disagreement. Pulse numbering can continue from captured timestamps, subject to the observation-gap and rounding limits described next.

Clock drift and limits

Pulse numbering uses the nominal ratio of **156.25 timer ticks per sample**. The actual ratio can differ slightly because the digitizer and capture timer run from different clocks.

For example, the elapsed tick count divided by 156.25 might give **1024.02** rather than exactly 1024. Rounding gives 1024 elapsed pulses. The difference between the calculated value and the nearest whole number - here, **0.02** - is called the *rounding residual*.

Software faults and stops recording if either:

- The time between successive observations exceeds **10 ms**.
- The rounding residual reaches **0.25 sample**.

Thus, 1024.02 passes the rounding check; 1024.26 fails it. The 0.25 limit leaves margin before the half-sample boundary where rounding would select a different pulse number.

Frequent observations limit how much clock drift affects each calculation. These checks protect pulse numbering; they do not measure or correct clock drift.

Clock-rate correction for sample times is performed separately during readback, using the measured FSYNC and PPS tick differences stored in each tBlock.

Relating samples to PPS and choosing block boundaries

When PPS rises at GPIO6, MCPWM saves the timer tick count. The PPS interrupt handler copies that saved timestamp for later use.

Software running on Core 0 then reads FSYNC timestamps until it obtains **two consecutive FSYNC pulses**. Their timestamps must differ by 156 or 157 timer ticks, corresponding to one sample period.

These two pulses establish the boundary between sBlocks:

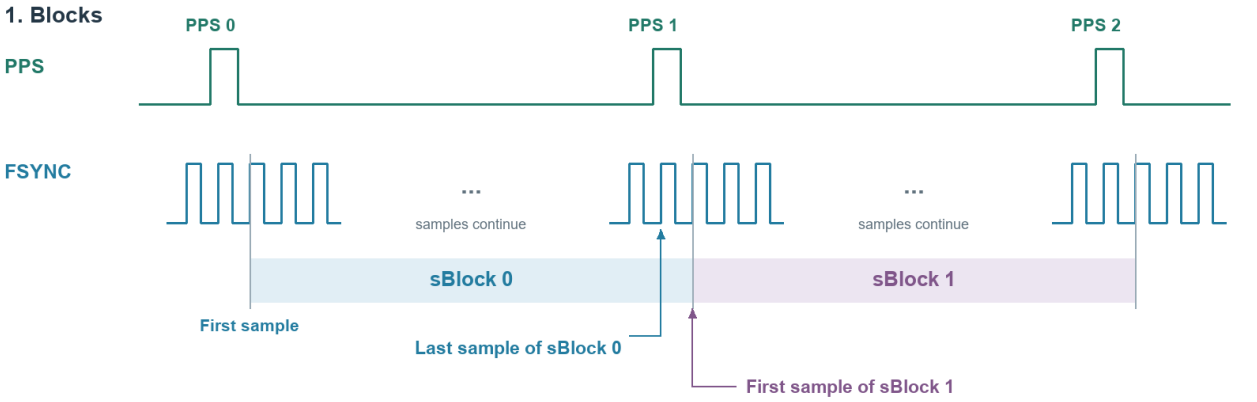
- The earlier pulse identifies the **last sample of the previous sBlock**.
- The later pulse identifies the **first sample of the next sBlock**.

Software determines their sample numbers using the recent numbered FSYNC reference described above.

The boundary occurs shortly **after PPS**, not necessarily at the first sample after PPS. This is acceptable because the file stores the actual PPS and FSYNC timestamps; their time difference is included in the sample-time calculation.

Consecutive sBlocks contain consecutive samples, with no duplicated or omitted sample at the boundary.

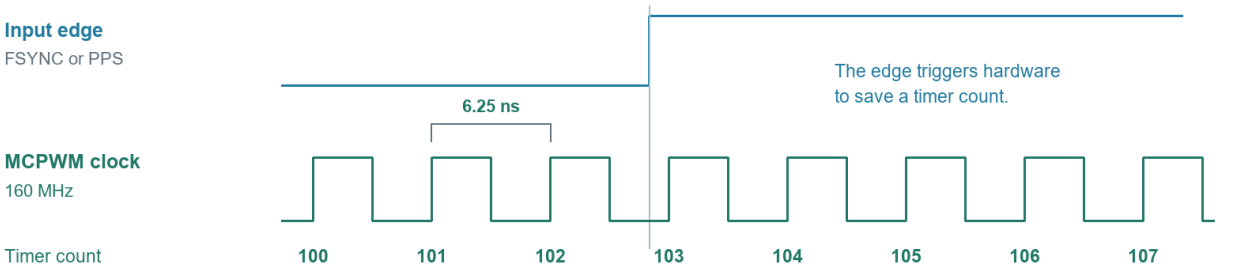
PPS, sample boundaries and the capture clock



The block split is shortly after PPS. The last and first samples are consecutive.

2. Inside the capture timer - enlarged time scale

FSYNC and PPS edges need not line up with the capture clock.



The timer advances in whole ticks; software reads the saved timestamp later.

The boundary search allows up to 32 reads and must finish in less than 20 µs. Both the numbered FSYNC reference and the PPS must be no more than 10 ms old when used for the boundary. If a valid boundary cannot be established within these limits, recording faults and stops.

Constructing a tBlock

A complete sBlock requires two boundaries: one following its opening PPS and one following its closing PPS.

The opening boundary supplies the first sample's FSYNC timestamp and sample number. The closing boundary supplies the last sample's FSYNC timestamp and the next block's first sample number.

Software fills the tBlock as follows:

Field	Value
`tcFsFirst`	FSYNC timestamp of this block's first sample
`tcFsLast`	FSYNC timestamp of this block's last sample
`tcPpsFirst`	Opening PPS timestamp
`tcPpsLast`	Closing PPS timestamp
`sCount`	Next block's first sample number minus this block's first sample number
`nSec`	Opening PPS's whole-second offset from the file's UTC starting time

For example, if this block starts at sample **1,000** and the next block starts at sample **1,025,000**, this block contains samples **1,000 through 1,024,999**:

```
sCount = 1,025,000 - 1,000 = 1,024,000 samples
```

The tBlock and its sBlock are ready for writing only after both boundaries and all the intervening samples are available.

Buffering samples for SD writing

Samples continue arriving while software waits for the closing PPS and establishes the block boundary. A **6-MiB staging buffer** holds those samples until the complete tBlock/sBlock pair can be assembled.

Software then transfers the pair into a separate **16-MiB ring buffer**. The disk-writing task takes data from this buffer and writes it to SD in **128-KiB chunks**.

These buffers allow acquisition to continue during temporary SD write delays. The saved timestamps describe the signal edges, so waiting in a buffer does not change a sample's calculated time.

If a buffer fills and incoming data cannot be retained, recording faults and stops rather than silently losing or overwriting samples.

Extending the timer count

The hardware capture timer uses a 32-bit counter. At 160 MHz, it reaches its maximum value and returns to zero approximately every **26.84 seconds**. This is called *wraparound*.

Software tracks these wraps through the frequent FSYNC observations and extends the timestamps into **64-bit tick counts**. PPS and block-boundary timestamps are placed on the same extended timeline.

For example:

```
Hardware count:  4,294,967,295 → 0
Extended count:  4,294,967,295 → 4,294,967,296
```

The tBlocks store the extended counts, so time calculations continue across hardware wraparound. The timer's starting value need not represent UTC: the PPS reference supplies the relationship between tick counts and absolute time.

Detecting missing or unexpected PPS pulses

Software checks the time between successive PPS pulses. The current limits accept intervals from **158,400,000 to 161,600,000 timer ticks**, equivalent to approximately **0.99-1.01 seconds** at the nominal timer frequency.

An interval outside those limits causes a fault. If no new PPS arrives within **176,000,000 ticks**, approximately **1.1 seconds**, software declares the pulse missing and stops recording.

These checks detect missing pulses or substantially incorrect pulse spacing. They do not establish that the GPS has a valid time solution or that its UTC message identifies the correct PPS pulse.